

CAPABILITY-BOUNDED CODE EVOLUTION WITH LARGE LANGUAGE
MODELS
A CROSS-FAMILY STUDY OF ADAPTIVE HEURISTIC SEARCH

by

FABIO COZZUTO

A dissertation submitted to the
Computer Science
in conformity with the requirements for
the degree of Master of Science

Bishop's University
Canada
June 2026

Copyright © Fabio Cozzuto, 2026
released under a [CC BY-SA 4.0 License](#)

Abstract

Large language models can now write executable code, repair programs, and participate in evaluator-guided search loops. These capabilities raise a concrete research question for computer science: can a language-model-driven code-evolution loop produce useful heuristic programs beyond one-shot generation, and under what conditions do apparent improvements reflect genuine adaptation rather than extra search budget or benchmark idiosyncrasies? This dissertation studies that question through a unified experimental framework in which models iteratively propose deterministic Python heuristics, receive structured feedback from task-specific validators, and are compared under replicated conditions on increasingly realistic problem families. The framework begins with simple adversarial games, then moves to benchmark combinatorial optimization tasks based on symmetric and asymmetric traveling salesperson problems and capacitated vehicle routing. Across these settings, the dissertation evaluates replay-aware memory mechanisms, modular operator discovery, adaptive heuristic portfolios, and budget-matched controls intended to separate genuine evolutionary effects from additional sampling. The evidence supports a capability-bounded account rather than broad algorithm-discovery dominance. The code-evolution loop reliably produces diverse executable heuristic variants and can preserve useful refinements when the benchmark leaves reachable heuristic headroom, but its apparent gains are strongly constrained by validator pressure, baseline strength, and task-specific interaction structure. In the final budget-controlled factorial analysis, replay-specific mechanisms do not show robust added value over budget-matched independent search. The dissertation contributes a measurement framework for studying when LLM-guided code evolution adapts, when it churns, and which failure modes recur across domains.

Acknowledgments

The author thanks Dr. Russell Butler for his supervision, guidance, and feedback throughout this research.

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Research Objectives	2
1.3	Working Hypotheses	3
1.4	Formal Setup and Notation	4
1.5	Why This Question Matters	6
1.6	Dissertation Scope	7
1.7	Main Thesis	8
1.8	Research Contributions	8
1.9	Preview of Findings	9
1.10	Organization of the Dissertation	10
2	Literature Review	11
2.1	Heuristics, Hyper-Heuristics, and Algorithm Selection	11
2.2	Formal Benchmark Problem Definitions	12
2.3	Machine Learning for Combinatorial Optimization	13
2.4	Large Language Models for Code Generation	14
2.5	Evaluator-Guided Search and Algorithm Discovery	15
2.6	Curriculum, Replay, and Diversity Mechanisms	15
2.7	Methodological Discipline and the Need for Conservative Claims . .	17
2.8	Synthesis and Research Gap	17
2.9	Chapter Summary	19
	Bibliography	20

List of Tables

1.1	Core notation used throughout the introduction and later methodological chapters.	5
1.2	Problem families covered by the dissertation and the corresponding evaluation focus.	7
2.1	Positioning of the present dissertation relative to adjacent literatures. The thesis sits closest to evaluator-guided program search, but its experimental logic also depends on benchmarked routing, hyper-heuristic reasoning, and conservative empirical methodology.	18

List of Figures

1.1	Conceptual structure of the LLM-guided code-evolution loop studied in this dissertation. The research question is not whether a single prompt can yield a useful heuristic, but whether the closed loop between archive, validator, and proposal model produces reliable adaptation under controlled budgets.	6
1.2	Representative held-out results from completed project phases. Lower is better in both panels. In the TSP replay suite, random replay achieved the best mean transfer gap among the replay variants. In the real-world CVRP suite, solver evolution improved over weaker bounded baselines yet remained behind the fixed Clarke–Wright savings heuristic on the primary endpoint.	9
2.1	Conceptual synthesis of the research threads integrated by the dissertation. The gap is not any single ingredient in isolation, but the absence of a cross-family, budget-controlled account of how these ingredients interact when the object of search is executable heuristic code.	19

Chapter 1

Introduction

The design of effective heuristics remains one of the central practical problems in computer science. Many computational tasks of scientific and industrial importance, including routing, scheduling, and resource allocation, are too expensive to solve exactly at the scales demanded by real applications. As a result, practitioners routinely rely on hand-crafted heuristics, metaheuristics, or solver portfolios whose quality depends heavily on human insight, empirical tuning, and domain knowledge [25, 5, 8, 12]. The question of how to automate some portion of this design process has therefore persisted across operations research, machine learning, and evolutionary computation for decades.

Recent progress in large language models (LLMs) changes the form of this question. Modern code models can synthesize non-trivial executable programs from natural-language specifications, solve benchmark programming tasks, and improve performance through repeated sampling and automated testing [9, 3, 18, 33, 17, 16]. More importantly for optimization, these models can be embedded inside outer loops that evaluate candidate programs, retain strong variants, and request improved descendants. Systems such as AlphaDev and FunSearch show that evaluator-guided search can go beyond naive one-shot completion when the problem admits fast verification and a suitably structured search space [21, 26]. At the same time, early work on LLMs as evolutionary optimizers suggests both promise and sharp scale limitations, especially when the search is applied to combinatorial optimization rather than small synthetic benchmarks [20, 27].

These developments motivate the present dissertation. The core problem is not whether an LLM can write *some* code for an optimization task. The more important scientific question is whether an iterative code-evolution loop can produce reliable, generalizable, and interpretable heuristic improvements under controlled experimental conditions. That question immediately raises harder sub-questions. If an iterative loop seems stronger than one-shot generation, is the gain due to a genuine memory or replay mechanism, or merely to spending more candidate budget? If a replay archive helps on one benchmark family, does the same effect survive on

other families with different structure and stronger validators? When a generated heuristic appears novel, does the novelty correspond to meaningful behavioural change, or only to superficial code churn? Finally, what happens when strong classical baselines already capture most of the available benchmark headroom?

1.1 Problem Statement

This dissertation studies LLM-guided code evolution as a *measurable adaptive search loop* rather than as a general claim of autonomous algorithm invention. The framework considered here is simple by design. A model receives a constrained programming interface and task feedback, proposes a deterministic Python heuristic, and then receives structured information about the heuristic’s behaviour and performance. The loop repeats over many epochs, storing code variants, validator signals, and archive summaries that can be replayed into later prompts. The scientific objective is to determine when such a loop adapts in a useful way and when it merely produces syntactically different but functionally unimportant variants.

The work therefore focuses on the following high-level research question:

Under what conditions does an LLM-guided code-evolution loop generate useful heuristic adaptation, and when are apparent gains better explained by search budget, benchmark headroom, validator pressure, or baseline strength?

This broad question is instantiated through several narrower empirical questions distributed across the dissertation:

1. Do replay-aware memory mechanisms improve held-out performance relative to no replay, random replay, and budget-matched controls?
2. Does the answer remain stable when the environment shifts from toy adversarial games to standard benchmark optimization families such as TSPLIB and CVRPLIB?
3. Can the search loop discover reusable heuristic components or adaptive controllers, rather than only full-program variants tied to a single scaffold?
4. How much of the observed performance variation is explained by base model strength, how much by the outer search procedure, and how much by their interaction with the target task family?

1.2 Research Objectives

The project is organized around four concrete objectives.

1. To formalize LLM-guided code evolution as a controlled experimental pipeline rather than an anecdotal prompt-engineering exercise.
2. To evaluate the pipeline on benchmark families whose structure and baseline difficulty differ materially, namely simple games, TSP, ATSP, and CVRP.
3. To distinguish replay-specific or memory-specific gains from generic benefits due to larger candidate budgets, broader search, or weaker baselines.
4. To produce a conservative account of the present capabilities and limits of executable heuristic evolution by LLMs, including null results and bounded wins.

1.3 Working Hypotheses

The hypotheses below state the principal empirical claims examined in this dissertation. They are framed to distinguish mechanism-specific effects from broader effects due to search budget, benchmark structure, and baseline strength.

1. **H1: Adaptive search under validator feedback.** When a benchmark family contains reachable heuristic headroom and the interface remains executable under repeated sampling, an iterative LLM-guided loop will generate non-trivial behavioural diversity and occasionally improve held-out quality over one-shot sampling.
2. **H2: Replay requires budget-matched evidence.** A replay or archive mechanism should be credited only if it improves held-out performance relative to a control with the same candidate budget and evaluation protocol.
3. **H3: Gains shrink under stronger evaluators.** Apparent advantages of evolved code should weaken as validator pressure rises, feasibility constraints tighten, and classical baselines consume more of the available benchmark headroom.
4. **H4: Transfer is partial rather than universal.** Search components discovered on one family may transfer across related problems, but the strength of that transfer should depend on the interaction structure of the target domain rather than on a task-agnostic notion of “general algorithmic intelligence.”

These hypotheses permit negative or mixed outcomes. Under controlled evaluation, null results are informative because they help separate genuine mechanism effects from artefacts of sampling budget or benchmark choice.

1.4 Formal Setup and Notation

The dissertation treats heuristic search as a problem of learning or discovering *programs* that map benchmark instances to feasible solutions. Let $\mathcal{X}$ denote a family of problem instances, let $\Omega(x)$ denote the feasible solution set for instance $x \in \mathcal{X}$, and let $f(\cdot; x)$ denote a minimization objective on $\Omega(x)$.

Definition 1.1. A benchmark optimization family is a triple $\mathcal{F} = (\mathcal{X}, \Omega, f)$ consisting of an instance distribution or benchmark set $\mathcal{X}$, a feasibility map Ω , and an objective function f to be minimized.

Definition 1.2. A deterministic heuristic program for $\mathcal{F}$ is an executable mapping $h : \mathcal{X} \rightarrow \Omega(x) \cup \{\perp\}$ that returns either a feasible solution or a detectable failure symbol $\perp$ under a fixed interface and resource budget.

This definition covers the constructive game agents of the early phases, the TSP and ATSP tour-construction heuristics of the routing phases, and the bounded CVRP solvers evolved in the later real-world experiments. The relevant distinction is whether a program yields a measurable objective value under controlled execution.

For any evaluation panel $\mathcal{D} \subseteq \mathcal{X}$, the empirical quality of a heuristic program is summarized by an aggregate metric

$$J_{\mathcal{D}}(h) = \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \text{gap}(h, x), \quad (1.1)$$

where lower values are better and the instance-level gap is defined, whenever a reference cost $f^*(x)$ is available, by

$$\text{gap}(h, x) = \frac{f(h(x); x) - f^*(x)}{f^*(x)}. \quad (1.2)$$

For feasibility-constrained settings such as CVRP, the implementation can replace f by a penalized objective $\tilde{f}$ that incorporates route-capacity or constraint violations. The form of (1.2) is standard in routing benchmarks because it allows comparisons across instances of different scales.

Definition 1.3. An LLM-guided code-evolution loop is an iterative process over a program space $\mathcal{H}$ with prompt state p_t , archive state $\mathcal{A}_t$, and evaluator E , such that

$$h_t \sim q_{\phi}(\cdot \mid p_t), \quad (1.3)$$

$$r_t = E(h_t; \mathcal{D}_{\text{train}}), \quad (1.4)$$

$$\mathcal{A}_t = U(\mathcal{A}_{t-1}, h_t, r_t), \quad (1.5)$$

$$p_{t+1} = C(\mathcal{T}, \mathcal{A}_t, r_t), \quad (1.6)$$

where q_{ϕ} is the model-conditioned program generator, U is the archive update rule, and C constructs the next prompt from the task specification $\mathcal{T}$ and archived evidence.

Symbol	Meaning
$\mathcal{X}$	benchmark family or instance space
$x \in \mathcal{X}$	a specific optimization instance
$\Omega(x)$	feasible solution set for instance x
$f(\cdot; x)$	instance-level minimization objective
$J_{\mathcal{D}}(h)$	aggregate evaluation metric of heuristic h on panel $\mathcal{D}$
$\mathcal{H}$	constrained program space searched by the outer loop
q_{ϕ}	model-conditioned proposal distribution over candidate programs
$\mathcal{A}_t$	archive or replay state available at iteration t
E	validator and scorer used to evaluate a candidate program
$\nu(h_t, \mathcal{A}_{t-1})$	archive-relative novelty diagnostic for candidate h_t

Table 1.1: Core notation used throughout the introduction and later methodological chapters.

Equations (1.3)–(1.6) make the research claim precise: replay is not a vague intuition, but a specific modification of the archive state $\mathcal{A}_t$ and prompt-construction operator C . A replay mechanism has independent value only if it improves final held-out performance relative to a control with a matched candidate budget. That comparison can be expressed as

$$\Delta_{\text{replay}} = J_{\mathcal{D}_{\text{test}}}(h^{\text{replay}}) - J_{\mathcal{D}_{\text{test}}}(h^{\text{budget-matched}}), \quad (1.7)$$

where a robustly negative value of Δ_{replay} would support a replay-specific advantage on a lower-is-better metric.

To separate meaningful behavioural diversity from mere lexical change, the loop also tracks novelty-like signals. A generic archive-relative novelty score can be written as

$$\nu(h_t, \mathcal{A}_{t-1}) = 1 - \max_{h \in \mathcal{A}_{t-1}} \text{sim}(h_t, h), \quad (1.8)$$

where sim is a code- or behaviour-level similarity measure. No single novelty metric is assumed to be canonical; novelty is treated as a diagnostic that must be interpreted jointly with validator outcomes and held-out quality.

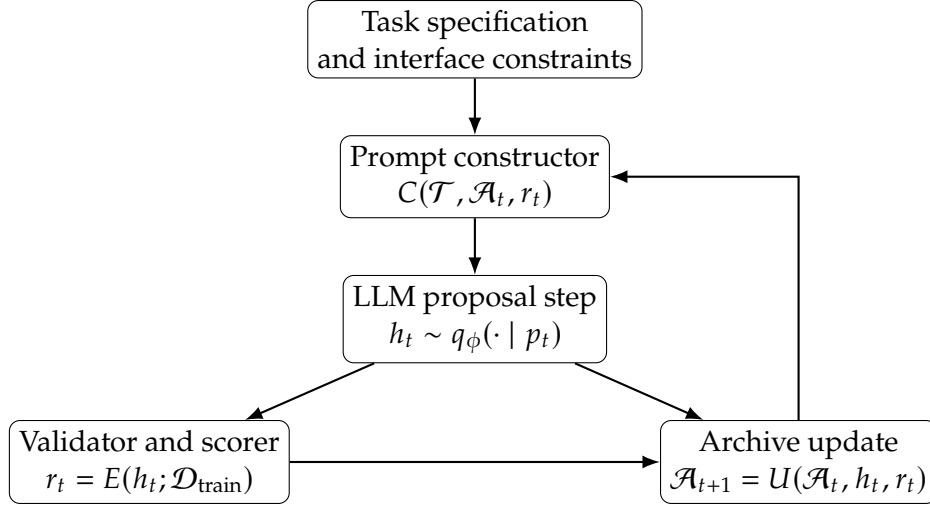


Figure 1.1: Conceptual structure of the LLM-guided code-evolution loop studied in this dissertation. The research question is not whether a single prompt can yield a useful heuristic, but whether the closed loop between archive, validator, and proposal model produces reliable adaptation under controlled budgets.

1.5 Why This Question Matters

The practical motivation follows directly from the role of heuristics in large-scale optimization. If language models can participate productively in heuristic search, then they could become useful assistants for constructing bounded optimization procedures in domains where exact methods are too slow and hand-designed heuristics are expensive to develop. The scientific motivation is subtler. Much current discussion around LLM-based scientific or algorithmic discovery emphasizes headline positive examples: competitive programming performance, small algorithmic improvements, or evaluator-guided discoveries in specialized settings [18, 21, 26]. Those results are important, but they do not by themselves establish a general understanding of when an LLM-driven search loop should work, when it should fail, and which controls are necessary to distinguish true adaptation from a larger sampling budget.

This issue is especially important in stochastic learning systems, where optimistic conclusions can be driven by best-run anecdotes, fragile benchmarks, or insufficient controls. Empirical machine learning has already seen the cost of such under-specified reporting, particularly in reinforcement learning, where replicated evaluation, uncertainty-aware summaries, and strong baselines are now widely recognized as necessary for credible claims [15, 1]. The same discipline is needed here. A dissertation on LLM-guided heuristic evolution should not only report positive cases. It should also identify limits, null results, and failure mechanisms.

Family	Benchmark basis	Evolved artifact	Primary held-out metric
Simple games	synthetic adversarial environments	deterministic game-playing policy code	win rate / score margin
TSP	TSPLIB symmetric routing instances	tour-construction or improvement heuristic code	relative optimality gap
ATSP	TSPLIB asymmetric routing instances	transfer-tested heuristic code	relative optimality gap
CVRP	CVRPLIB, especially Uchoa X instances	whole solver or route-construction code	penalized gap with feasibility pressure

Table 1.2: Problem families covered by the dissertation and the corresponding evaluation focus.

1.6 Dissertation Scope

The work reported here begins with simple adversarial games and then moves to increasingly realistic combinatorial optimization families. This progression is deliberate. The initial game environments provide a cheap setting in which iterative adaptation, curriculum effects, replay, and behavioural diversity can be observed directly. Later phases retain the same basic code-evolution machinery but replace the toy environment with benchmark-driven tasks whose solution quality can be evaluated against established references and classical heuristics. In particular, the dissertation studies:

1. adversarial code evolution in simple two-player games;
2. replay-aware heuristic evolution on symmetric TSPLIB traveling salesperson benchmarks;
3. transfer of the same design ideas to asymmetric TSPLIB ATSP and CVRPLIB CVRP families;
4. modular operator discovery and adaptive heuristic portfolio control on TSP;
5. bounded whole-solver evolution on real-world CVRPLIB X instances; and
6. a final crossed factorial analysis that separates model strength, search budget, and replay-specific mechanisms.

The study therefore extends beyond a single TSP setting, but it does not attempt to formulate a universal theory of automatic algorithm discovery. Its aim is narrower: to develop a unified framework for studying *capabilities and limits* in LLM-guided code evolution across domains with different structural demands.

1.7 Main Thesis

The central thesis of this dissertation is as follows:

Across simple games, TSP, ATSP, and real-world CVRP, LLM-guided code evolution can generate diverse executable heuristic programs and sometimes preserve useful adaptations, but its success is capability-bounded. In the current evidence, performance depends strongly on validator pressure, baseline strength, benchmark headroom, and task-specific interaction structure. Replay-specific mechanisms do not exhibit robust added value once they are compared against budget-matched independent search.

The evidence supports a conservative interpretation. It does not support two stronger narratives: that replay uniformly improves performance, or that iterative LLM search has already surpassed mature human-designed optimization heuristics in a general sense. Instead, the empirical record supports a framework for measuring where adaptation occurs, where it fails to transfer, and how to separate meaningful improvement from extra budget and superficial novelty.

1.8 Research Contributions

The dissertation makes five main contributions.

1. It introduces a unified experimental framework for iterative code evolution with archived feedback, constrained execution, deterministic validators, and report-first research artifacts.
2. It extends the study of replay-aware code evolution from toy adversarial settings to multiple benchmark optimization families, including symmetric TSP, asymmetric TSP, and capacitated vehicle routing.
3. It evaluates not only positive search variants but also mechanism-focused alternatives, including curated failure replay, compression pressure, modular operator evolution, portfolio-based selection, and budget-matched no-replay controls.
4. It emphasizes replicated, uncertainty-aware, benchmark-based evaluation in the spirit of careful empirical RL methodology [15, 1].
5. It contributes a conservative cross-family synthesis in which negative and boundary results are treated as findings, not as failures to be hidden.

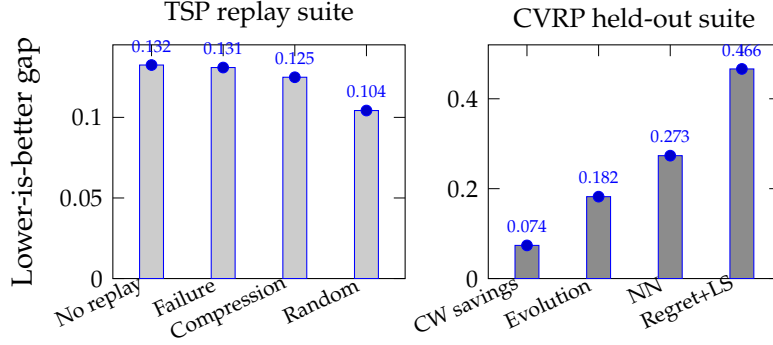


Figure 1.2: Representative held-out results from completed project phases. Lower is better in both panels. In the TSP replay suite, random replay achieved the best mean transfer gap among the replay variants. In the real-world CVRP suite, solver evolution improved over weaker bounded baselines yet remained behind the fixed Clarke–Wright savings heuristic on the primary endpoint.

1.9 Preview of Findings

The main empirical findings are mixed but coherent.

First, iterative code evolution reliably produces executable diversity. Across multiple phases, the system generates non-trivial program variation rather than collapsing immediately to a single artifact. Second, replay can matter, but not in the simple way initially hypothesized. On symmetric TSP, naive raw-failure replay is not the strongest mechanism; replay curation matters more than failure accumulation alone. Third, later higher-upside phases behave as strict falsification tests. Modular operator discovery produces genuine rediscovery structure, yet no operator survives the full validation pipeline. Adaptive portfolio control improves over a weak one-shot LLM selector, yet does not beat the strongest fixed heuristic when oracle headroom is absent. Whole-solver evolution on real-world CVRP stays feasible and improves over weaker bounded baselines, but not over the strongest fixed Clarke–Wright baseline. Finally, the terminal cross-family factorial shows that replay-specific mechanisms do not beat budget-matched no-replay controls with positive bootstrap support, and that pooled variance is dominated more by task-specific interaction structure than by a clean main effect of model strength.

Taken together, these findings support a *capability-and-limits* interpretation. The system is good enough to warrant serious study, but not strong enough to justify claims of general algorithm-discovery superiority.

1.10 Organization of the Dissertation

The remainder of the dissertation is organized as follows. Chapter 2 reviews the relevant literature on heuristics and algorithm selection, machine learning for combinatorial optimization, LLM-based code generation, evaluator-guided algorithm discovery, and curriculum, replay, and diversity mechanisms. Later chapters will present the experimental framework, the benchmark families, and the phase-by-phase empirical results. The concluding chapters will return to the main thesis and synthesize what the full evidence implies about the present capabilities and limits of LLM-guided code evolution.

Chapter 2

Literature Review

2.1 Heuristics, Hyper-Heuristics, and Algorithm Selection

The practical difficulty of many combinatorial optimization problems has long made heuristic design an essential part of computer science and operations research. Rice’s classic formulation of the algorithm selection problem cast the issue in its most general form: given a space of instances, a portfolio of candidate algorithms, and a performance measure, the task is to learn or construct a mapping from instances to algorithms that performs well in expectation [25]. In modern notation, if $\mathcal{X}$ is an instance family, $\mathcal{A}$ is a portfolio of algorithms, and $m(a, x)$ is a lower-is-better performance metric, then the ideal selector is

$$\pi^* \in \arg \min_{\pi: \mathcal{X} \rightarrow \mathcal{A}} \mathbb{E}_{x \sim \mathcal{D}} [m(\pi(x), x)]. \quad (2.1)$$

Equation (2.1) remains foundational because it separates the underlying optimization problem from the meta-level question of which procedure should be applied to which instance.

Hyper-heuristics emerged as one prominent response to this meta-level challenge. Rather than searching directly over candidate solutions, hyper-heuristics search over heuristics or heuristic components, with the goal of automating heuristic selection, sequencing, or generation [8, 12]. This shift from solution space to heuristic space is especially relevant for the present dissertation because the proposed LLM-guided loop also operates primarily at the level of *programs that produce solutions*, not at the level of solutions themselves. Recent surveys emphasize that hyper-heuristics now span selection, generation, portfolio, and configuration settings, and that modern work increasingly integrates machine learning into all four categories [12]. The dissertation can therefore be situated naturally within this tradition, with the difference that the search operator is a language model capable of proposing new executable code rather than a classical mutation, grammar, or rule-combination mechanism.

Algorithm selection has likewise evolved toward both feature-based and feature-free approaches. Traditional methods rely on hand-designed instance descriptors, whereas newer methods attempt to learn directly from raw instance representations [2]. This distinction becomes important in later chapters, especially for the adaptive heuristic portfolio experiments, because it frames a key design choice: should a controller rely on explicit descriptors crafted from domain knowledge, or can a more generic representation learn the selector implicitly?

2.2 Formal Benchmark Problem Definitions

The empirical phases of this dissertation are built around standard benchmark optimization families. A concise formal account is therefore useful before turning to learning-based methods.

2.2.1 Symmetric Traveling Salesperson Problem

In the symmetric traveling salesperson problem (TSP), one is given a complete weighted graph $G = (V, E, w)$ with $w_{ij} = w_{ji} \geq 0$ and seeks a minimum-cost Hamiltonian cycle [24]. Using binary edge-selection variables x_{ij} , a standard formulation is

$$\min_x \sum_{(i,j) \in E} w_{ij} x_{ij} \quad (2.2)$$

$$\text{s.t.} \quad \sum_{j \neq i} x_{ij} = 2, \quad \forall i \in V, \quad (2.3)$$

$$\sum_{i,j \in S, i < j} x_{ij} \leq |S| - 1, \quad \forall S \subset V, 2 \leq |S| < |V|, \quad (2.4)$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in E. \quad (2.5)$$

Constraint (2.3) enforces degree two at every city, while (2.4) removes disconnected subtours. The TSP is NP-hard, which explains why heuristic methods remain central despite decades of exact and approximate progress.

2.2.2 Asymmetric Traveling Salesperson Problem

The asymmetric TSP (ATSP) replaces the undirected graph by a complete directed graph $G = (V, A, c)$ in which c_{ij} and c_{ji} may differ. The goal is again to find a minimum-cost Hamiltonian tour, but now with one incoming and one outgoing arc per city. Many constructive ideas transfer from TSP to ATSP, yet the asymmetry changes the local-search geometry and often weakens heuristics that rely on symmetric exchange structure. This makes ATSP a useful transfer family for testing

whether a learned or evolved heuristic reflects a genuine search principle rather than a benchmark-specific shortcut.

2.2.3 Capacitated Vehicle Routing Problem

In the capacitated vehicle routing problem (CVRP), a depot node 0, customer set $V = \{1, \dots, n\}$, customer demands $q_i > 0$, and vehicle capacity Q are given. The task is to partition customers into depot-rooted routes whose total demand does not exceed Q and whose combined travel cost is minimal [10, 31]. A compact route-based view is

$$\min_{\mathcal{R}} \sum_{R \in \mathcal{R}} c(R) \quad (2.6)$$

$$\text{s.t. } \bigcup_{R \in \mathcal{R}} R = V, \quad R \cap R' = \emptyset \text{ for } R \neq R', \quad (2.7)$$

$$\sum_{i \in R} q_i \leq Q, \quad \forall R \in \mathcal{R}. \quad (2.8)$$

Unlike TSP, CVRP combines sequencing and packing structure. That added constraint pressure matters for this dissertation because a program that looks promising under tour construction alone may fail once feasibility management, route merging, and penalty handling become part of the evaluator.

2.3 Machine Learning for Combinatorial Optimization

Machine learning for combinatorial optimization has developed into a substantial field in its own right. Bengio, Lodi, and Prouvost provide one of the clearest organizing views: optimization problems can be treated as data points, and learning can be used to replace costly or poorly defined decisions inside larger solving procedures [5]. This perspective is especially influential because it avoids the misleading idea that learning must replace classical optimization wholesale. Instead, machine learning can support branching, ranking, repair, construction, selection, or parameterization within a broader pipeline.

One important research stream studies direct learning-based solvers. Neural combinatorial optimization with reinforcement learning showed early promise on Euclidean TSP by training neural policies to construct tours directly from coordinates [4]. Khalil et al. extended this direction by learning greedy algorithms over graph-structured state representations, showing that learned policies can act as reusable meta-algorithms rather than as fixed instance-specific predictors [11]. Subsequent work extended learning-based optimization to more constrained settings and more realistic objectives, but the broader lesson remained the same: learned solvers can be effective, yet strong classical heuristics often remain difficult to beat across wide

instance distributions [5]. This tension matters for the current dissertation, which does not ask an LLM to output one final tour directly. Instead, it asks whether an LLM can iteratively improve the *code* of a heuristic solver under benchmark feedback.

The routing literature also provides the baselines and datasets that make such claims meaningful. TSPLIB remains the canonical reference library for traveling salesperson problems [24]. For symmetric TSP, the Lin–Kernighan family and its refinements, including Helsgaun’s implementation, remain among the strongest and most influential heuristic baselines [19, 14]. For CVRP, Clarke and Wright’s savings heuristic remains a foundational constructive baseline [10], while modern state-of-the-art practice includes far stronger hybrid metaheuristics such as Vidal’s hybrid genetic search [32]. The modern Uchoa instance family likewise provides a more challenging and statistically informative benchmark regime than many older CVRP sets [31]. Together, these references define the level of baseline strength against which LLM-evolved heuristics must be judged. A language-model system that only beats naive constructive baselines should not be conflated with one that beats the best established heuristics in the literature.

2.4 Large Language Models for Code Generation

The empirical framework studied in this dissertation depends directly on the rapid improvement of code-capable language models. Codex demonstrated that large language models trained on code could solve a meaningful fraction of functional programming tasks and that repeated sampling substantially boosts success rates on harder problems [9]. Austin et al. likewise showed that program synthesis performance scales strongly with model size and that natural-language feedback can materially improve generated programs [3]. These findings indicate that outer-loop search and repair are integral components of strong performance on demanding code-generation tasks.

As the field matured, surveys began to distinguish between simple code completion, benchmarked program synthesis, repair, debugging, and agentic multi-step workflows. Early overviews of NL2Code models emphasized training data, scaling, and benchmark design [33], while more recent surveys document a much larger and more heterogeneous landscape of code models, evaluation protocols, and practical risks [17]. At the same time, stronger benchmarks such as AlphaCode-style competitive programming and LiveCodeBench reveal that the difference between solving simple textbook-style tasks and solving harder, more recent coding problems remains substantial [18, 16]. This distinction reinforces the need to evaluate code generation under task-specific execution and held-out tests, rather than by surface plausibility alone.

2.5 Evaluator-Guided Search and Algorithm Discovery

One-shot code generation is only one end of the design spectrum. A more relevant line of work for this dissertation combines generative models with evaluators that score candidate programs and feed the results back into an outer search loop. In abstract form, the objective is no longer to predict source code that merely “looks right,” but to search a program space $\mathcal{H}$ for

$$h^* \in \arg \min_{h \in \mathcal{H}} J_{\mathcal{D}}(h), \quad (2.9)$$

where $J_{\mathcal{D}}$ is defined by measurable performance on a benchmark panel. This framing is important because it aligns the generative model with verification rather than with free-form textual plausibility.

FunSearch is a central example. It pairs a frozen LLM with a systematic evaluator and a diverse program archive in order to discover high-scoring programs for problems that are hard to solve but easy to evaluate [26]. Its importance lies not only in its results, but in its formulation: the LLM is used as a variation operator inside an evolutionary process, while the evaluator guards against incorrect or hallucinated proposals.

AlphaDev demonstrates a related idea from a reinforcement-learning perspective. It formulates algorithm discovery for low-level sorting routines as a single-player game and optimizes directly for correctness and latency at the assembly level [21]. AlphaCode, while not an evolutionary system in the same sense, also depends heavily on large-scale sampling, behaviour-based filtering, and selection among many candidate programs [18]. Together, these systems suggest a broader pattern: modern code-generating AI systems often rely on an interaction between generative diversity and external verification, not on a single perfect sample.

The most direct predecessor to the present dissertation in combinatorial optimization is the study of LLMs as evolutionary optimizers by Liu et al. [20]. Their system treats the LLM as a crossover-and-mutation operator within a population-based optimizer and shows promising TSP results on very small instances. This work is conceptually important, but its scale also exposes a gap. There remains a need for research that studies executable whole-program heuristic evolution on standard benchmark families, with stronger baselines, repeated trials, transfer evaluation, and mechanism controls. A recent systematic review of LLMs in combinatorial optimization confirms both the rapid expansion of the area and the diversity of roles LLMs now play in optimization systems [27]. However, the review also implies that careful cross-family empirical evaluation remains comparatively rare.

2.6 Curriculum, Replay, and Diversity Mechanisms

The replay-aware components of this dissertation draw from several adjacent literatures rather than from one single source. Curriculum learning offers one

motivation. Bengio et al. formalized the idea that ordering examples from easier to harder can improve learning and generalization [6]. Later surveys and automated curriculum methods extended the idea to a wide range of machine learning and reinforcement learning settings [30, 13]. In adversarial or sequential domains, curricula matter because the learner’s exposure distribution is part of the effective training algorithm.

Replay mechanisms supply a second motivation. In reinforcement learning, prioritized experience replay showed that non-uniform reuse of past transitions can improve learning efficiency relative to uniform replay [28]. In one common formulation, transition i is drawn with probability

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}, \quad (2.10)$$

where p_i is a priority score and α controls the strength of prioritization. Later work generalized this idea to sequence-level replay and more structured replay decisions [7]. Although the present dissertation does not replay transitions in the standard RL sense, it reuses *instances*, failure cases, and archive summaries as a memory mechanism for future program proposals. That analogy is strong enough to be informative, but weak enough that it must be tested rather than assumed. A replay strategy that helps temporal-difference learning does not automatically imply a successful replay strategy for program-level heuristic evolution.

Diversity-based search provides a third motivation. Quality-diversity methods such as MAP-Elites and novelty-search-with-local-competition aim not merely to optimize a single objective, but to retain a collection of high-performing, behaviourally distinct artifacts [22, 23]. This family of ideas is relevant because a code-evolution loop can easily collapse to narrow local search unless the archive preserves multiple useful regions of the behavioural space. The present dissertation therefore draws on QD-style intuitions when it tracks novelty, maintains archives, and evaluates whether diversity corresponds to genuine adaptation rather than lexical churn.

Finally, self-play and adversarial practice provide a motivating example for why curriculum and iterative pressure can induce capability growth at all. AlphaGo Zero showed that high-level performance can emerge from repeated self-play without human demonstrations [29]. Its scale and claims differ substantially from those considered here. Nevertheless, the underlying intuition remains relevant: iterative interaction can sometimes unlock behaviour that is hard to obtain from static supervision alone.

2.7 Methodological Discipline and the Need for Conservative Claims

An important part of the literature relevant to this dissertation concerns *how* claims are validated, not only *what* systems are built. Henderson et al. document the reproducibility problems that arise when stochastic learning systems are evaluated with too few seeds, weak statistical reporting, or poorly matched baselines [15]. Agarwal et al. extend this point by arguing for robust statistical summaries and uncertainty-aware comparisons in deep RL benchmarking [1]. Although these works target reinforcement learning rather than code evolution, the methodological lesson transfers directly. When a generative system can produce very different candidate programs across runs, best-case anecdotes are not enough.

This methodological literature motivates the use of paired offsets, aggregate reports, bootstrap comparisons, held-out panels, and budget-matched controls in the present study. A simple paired comparison between two conditions A and B can be written as

$$\hat{\Delta}_{A,B} = \frac{1}{n} \sum_{i=1}^n \left(m_i^{(A)} - m_i^{(B)} \right), \quad (2.11)$$

with uncertainty estimated by resampling or other repeated-evaluation procedures. A negative result obtained under proper controls can be more informative than a seemingly positive result that confounds memory mechanisms with extra candidate budget.

2.8 Synthesis and Research Gap

Table 2.1 summarizes the main research traditions from which this dissertation draws.

Taken together, the existing literature establishes five points.

1. Classical optimization already provides strong benchmark libraries and strong heuristic baselines [24, 19, 14, 10, 31, 32].
2. Hyper-heuristics and algorithm selection offer established ways to reason about search in heuristic space rather than solution space [25, 8, 12].
3. Modern code LLMs are strong enough to generate executable programs, especially when combined with repeated sampling, testing, and repair [9, 3, 18, 16].
4. Evaluator-guided algorithm discovery with AI is now credible, but existing success stories often depend on specialized evaluators, narrow scopes, or problem formulations that differ from full-program benchmark optimization [21, 26, 20].

Tradition	Search object	Main strength	Main limitation	Representative refs.
Classical heuristics and metaheuristics	solutions or route moves	strong benchmarked performance	expensive domain expertise	[19, 14, 10, 32]
Hyper-heuristics and algorithm selection	heuristic choices or generators	explicit heuristic-level reasoning	relies on crafted operators or features	[25, 8, 12]
Learning-based combinatorial optimization	policies, rankings, or solver components	learns recurring structure from data	difficult to beat strong classical solvers broadly	[5, 4, 11]
Evaluator-guided program search with LLMs	executable programs	open-ended code generation plus verification	noisy search; mechanism claims need strong controls	[26, 21, 20]

Table 2.1: Positioning of the present dissertation relative to adjacent literatures. The thesis sits closest to evaluator-guided program search, but its experimental logic also depends on benchmarked routing, hyper-heuristic reasoning, and conservative empirical methodology.

- Curriculum, replay, and diversity mechanisms are theoretically motivated, but their value in LLM-guided *code evolution* remains an empirical question rather than a settled fact [6, 28, 23].

The gap addressed by this dissertation lies at the intersection of these points. There is still no clear empirical account of how LLM-guided code evolution behaves across multiple benchmark families when evaluated with replicated runs, strong classical baselines, replay alternatives, and budget-matched controls. The dissertation’s purpose is to fill that gap with a conservative cross-family study centered on executable heuristics, not on isolated positive examples.

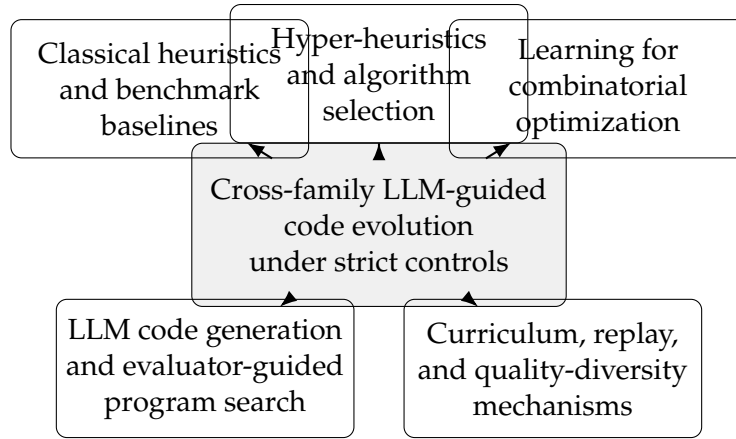


Figure 2.1: Conceptual synthesis of the research threads integrated by the dissertation. The gap is not any single ingredient in isolation, but the absence of a cross-family, budget-controlled account of how these ingredients interact when the object of search is executable heuristic code.

2.9 Chapter Summary

This chapter established the technical context for the dissertation in four steps. First, it positioned the work within algorithm selection and hyper-heuristics, where the central object of search is a heuristic procedure rather than a single candidate solution. Second, it reviewed the benchmark optimization families that make the empirical claims meaningful, including the standard TSP, ATSP, and CVRP formulations and their mature baseline literatures. Third, it situated the project within modern code-generation and evaluator-guided program-search research, where LLMs are most credible when paired with strong external verification. Finally, it argued that replay, curriculum, and diversity mechanisms must be evaluated under careful experimental discipline rather than inferred from isolated positive cases. The next methodological chapters can therefore build on a clear research gap: cross-family, budget-controlled, executable heuristic evolution remains insufficiently characterized in the literature.

Bibliography

- [1] R. AGARWAL, M. SCHWARZER, P. S. CASTRO, A. COURVILLE, AND M. G. BELLEMARE, *Deep reinforcement learning at the edge of the statistical precipice*, in Advances in Neural Information Processing Systems 35, 2021.
- [2] M. ALISSA, E. ÖZCAN, AND G. KENDALL, *Automated algorithm selection: from feature-based to feature-free approaches*, Journal of Heuristics, 29 (2023), pp. 1–38.
- [3] J. AUSTIN, A. ODENA, M. NYE, M. BOSMA, H. MICHALEWSKI, D. DOHAN, E. JIANG, C. CAI, M. TERRY, Q. LE, AND C. SUTTON, *Program synthesis with large language models*, arXiv preprint arXiv:2108.07732, (2021).
- [4] I. BELLO, H. PHAM, Q. V. LE, M. NOROUZI, AND S. BENGIO, *Neural combinatorial optimization with reinforcement learning*, arXiv preprint arXiv:1611.09940, (2016).
- [5] Y. BENGIO, A. LODI, AND A. PROUVOST, *Machine learning for combinatorial optimization: A methodological tour d’horizon*, European Journal of Operational Research, 290 (2021), pp. 405–421.
- [6] Y. BENGIO, J. LOURADOUR, R. COLLOBERT, AND J. WESTON, *Curriculum learning*, in Proceedings of the 26th International Conference on Machine Learning, 2009, pp. 41–48.
- [7] M. BRITAIN, J. BERTRAM, X. YANG, AND P. WEI, *Prioritized sequence experience replay*, arXiv preprint arXiv:1905.12726, (2019).
- [8] E. K. BURKE, M. GENDREAU, M. HYDE, G. KENDALL, G. OCHOA, E. ÖZCAN, AND R. QU, *Hyper-heuristics: A survey of the state of the art*, Journal of the Operational Research Society, 64 (2013), pp. 1695–1724.
- [9] M. CHEN, J. TWOREK, H. JUN, Q. YUAN, H. P. DE OLIVEIRA PINTO, J. KAPLAN, H. EDWARDS, Y. BURDA, N. JOSEPH, G. BROCKMAN, A. RAY, R. PURI, G. KRUEGER, M. PETROV, H. KHLAAF, G. SASTRY, P. MISHKIN, B. CHAN, S. GRAY, N. RYDER, M. PAVLOV, A. POWER, L. KAISER, M. BAVARIAN, C. WINTER, P. TILLET, F. P. SUCH, D. CUMMINGS, M. PLAPPERT, F. CHANTZIS, E. BARNES, A. HERBERT-VOSS, W. H. GUSS, A. NICHOL, A. PAINO, N. TEZAK, J. TANG, I. BABUSCHKIN, S. BALAJI, S. JAIN,

- W. SAUNDERS, C. HESSE, A. N. CARR, J. LEIKE, J. ACHIAM, V. MISRA, E. MORIKAWA, A. RADFORD, M. KNIGHT, M. BRUNDAGE, M. MURATI, K. MAYER, P. WELINDER, B. MCGREW, D. AMODEI, S. MCCANDLISH, I. SUTSKEVER, AND W. ZAREMBA, *Evaluating large language models trained on code*, arXiv preprint arXiv:2107.03374, (2021).
- [10] G. CLARKE AND J. W. WRIGHT, *Scheduling of vehicles from a central depot to a number of delivery points*, *Operations Research*, 12 (1964), pp. 568–581.
- [11] H. DAI, E. B. KHALIL, Y. ZHANG, B. DILKINA, AND L. SONG, *Learning combinatorial optimization algorithms over graphs*, in *Advances in Neural Information Processing Systems* 30, 2017.
- [12] T. DÖKEROGLU, T. KUCUKYILMAZ, AND E.-G. TALBI, *Hyper-heuristics: A survey and taxonomy*, *Computers & Industrial Engineering*, 187 (2024), p. 109815.
- [13] A. GRAVES, M. G. BELLEMARE, J. MENICK, R. MUNOS, AND K. KAVUKCUOGLU, *Automated curriculum learning for neural networks*, in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1311–1320.
- [14] K. HELSGAUN, *An effective implementation of the lin-kernighan traveling salesman heuristic*, *European Journal of Operational Research*, 126 (2000), pp. 106–130.
- [15] P. HENDERSON, R. ISLAM, P. BACHMAN, J. PINEAU, D. PRECUP, AND D. MEGER, *Deep reinforcement learning that matters*, *Proceedings of the AAAI Conference on Artificial Intelligence*, 32 (2018).
- [16] N. JAIN, K. HAN, A. GU, W.-D. LI, F. YAN, T. ZHANG, S. WANG, A. SOLAR-LEZAMA, K. SEN, AND I. STOICA, *LiveCodeBench: Holistic and contamination free evaluation of large language models for code*, arXiv preprint arXiv:2403.07974, (2024).
- [17] J. JIANG, F. WANG, J. SHEN, S. KIM, AND S. KIM, *A survey on large language models for code generation*, arXiv preprint arXiv:2406.00515, (2024).
- [18] Y. LI, D. CHOI, J. CHUNG, N. KUSHMAN, J. SCHRITTWIESER, R. LEBLOND, T. ECCLES, J. KEELING, F. GIMENO, A. D. LAGO, T. HUBERT, P. CHOY, C. DE MASSON D’AUTUME, I. BABUSCHKIN, X. CHEN, P.-S. HUANG, J. WELBL, S. GOWAL, A. CHEREPANOV, J. MOLLOY, D. J. MANKOWITZ, E. S. ROBSON, P. KOHLI, N. DE FREITAS, K. KAVUKCUOGLU, AND O. VINYALS, *Competition-level code generation with AlphaCode*, *Science*, 378 (2022), pp. 1092–1097.
- [19] S. LIN AND B. W. KERNIGHAN, *An effective heuristic algorithm for the traveling-salesman problem*, *Operations Research*, 21 (1973), pp. 498–516.
- [20] S. LIU, C. CHEN, X. QU, K. TANG, AND Y.-S. ONG, *Large language models as evolutionary optimizers*, in *2024 IEEE Congress on Evolutionary Computation (CEC)*, 2024, pp. 1–8.

- [21] D. J. MANKOWITZ, A. MICH, A. ZHERNOV, M. GELMI, M. SELVI, C. PADURARU, E. LEURENT, S. IQBAL, J.-B. LESPIAU, A. AHERN, T. KÖPPE, K. MILLIKIN, S. GAFFNEY, T. RASHID, Y. LI, D. B. DOHAN, H. PHAM, D. SILVER, K. KAVUKCUOGLU, P. KOHLI, AND O. VINYALS, *Faster sorting algorithms discovered using deep reinforcement learning*, *Nature*, 618 (2023), pp. 257–263.
- [22] J.-B. MOURET AND J. CLUNE, *Illuminating search spaces by mapping elites*, arXiv preprint arXiv:1504.04909, (2015).
- [23] J. K. PUGH, L. B. SOROS, AND K. O. STANLEY, *Quality diversity: A new frontier for evolutionary computation*, *Frontiers in Robotics and AI*, 3 (2016), p. 40.
- [24] G. REINELT, *TSPLIB—a traveling salesman problem library*, *ORSA Journal on Computing*, 3 (1991), pp. 376–384.
- [25] J. R. RICE, *The algorithm selection problem*, *Advances in Computers*, 15 (1976), pp. 65–118.
- [26] B. ROMERA-PAREDES, M. BAREKATAIN, A. NOVIKOV, M. BALOG, M. P. KUMAR, E. DUPONT, F. J. R. RUIZ, J. S. ELLENBERG, P. WANG, O. FAWZI, P. KOHLI, AND A. FAWZI, *Mathematical discoveries from program search with large language models*, *Nature*, 625 (2024), pp. 468–475.
- [27] F. D. ROS, M. SOPRANO, L. D. GASPERO, AND K. ROITERO, *Large language models for combinatorial optimization: A systematic review*, arXiv preprint arXiv:2507.03637, (2025).
- [28] T. SCHAUL, J. QUAN, I. ANTONOGLOU, AND D. SILVER, *Prioritized experience replay*, arXiv preprint arXiv:1511.05952, (2015).
- [29] D. SILVER, J. SCHRITTWIESER, K. SIMONYAN, I. ANTONOGLOU, A. HUANG, A. GUEZ, T. HUBERT, L. BAKER, M. LAI, A. BOLTON, Y. CHEN, T. LILLICRAP, F. HUI, L. SIFRE, G. VAN DEN DRIESSCHE, T. GRAEPEL, AND D. HASSABIS, *Mastering the game of go without human knowledge*, *Nature*, 550 (2017), pp. 354–359.
- [30] P. SOVIAN, R. T. IONESCU, P. ROTA, AND N. SEBE, *Curriculum learning: A survey*, *International Journal of Computer Vision*, 130 (2022), pp. 1526–1565.
- [31] E. UCHOA, D. PECIN, A. PESSOA, M. POGGI, T. VIDAL, AND A. SUBRAMANIAN, *New benchmark instances for the capacitated vehicle routing problem*, *European Journal of Operational Research*, 257 (2017), pp. 845–858.
- [32] T. VIDAL, *Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood*, *Computers & Operations Research*, 140 (2022), p. 105643.

- [33] D. ZAN, B. CHEN, F. ZHANG, D. LU, B. WU, B. GUAN, W. YONGJI, AND J.-G. LOU, *Large language models meet NL2Code: A survey*, in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 7443–7464.